9 Actor-Critic Methods

9.1 Q-Value based Actor-Critic (QAC) Algorithm

Q-Value based Actor-Critic (QAC): the Simplest Actor-Critic

首先回顾 Policy Gradient 的基本思想:

① 定义 scalar metric $J(\theta)$ 来衡量 policy π_θ 的好坏，其可以为 $\bar{v}_\pi = \sum_{s\in\mathcal{S}} d(s)v_\pi(s)$ 或 $\bar{r}_\pi \triangleq \sum_{s\in\mathcal{S}} d_\pi(s)r_\pi(s)$.

② 为获得 optimal policy 需要最大化 $J(\theta)$，即 $\theta^* = \arg\max_\theta J(\theta)$，需要使用梯度上升 (gradient ascent):

$$\theta_{t+1} = \theta_t + \alpha\nabla_\theta J(\theta_t) = \theta_t + \alpha\mathbb{E}_{s\sim d^\pi,\, a\sim\pi_\theta}\left[\nabla_\theta \log \pi_\theta(a \mid s, \theta_t) Q^\pi(s,a)\right]. \tag{9.1}$$

③ 实际中一般将理论上的期望转化为采样的 stochastic 版本:

$$\theta_{t+1} = \theta_t + \alpha\nabla_\theta \ln \pi(a_t|s_t,\theta_t) q_t(s_t,a_t). \tag{9.2}$$

在上述过程中实际存在两种角色: (1) **Actor:** 参数为 θ 的 policy π_θ，负责选择动作; (2) **Critic:** 负责评估 Actor 的表现，使用某种算法给出 $Q^\pi(s,a)$ 或 $q_t(s_t,a_t)$ 的估计值. 因此上述过程中 Actor 根据 Critic 提供的价值估计 q 按照式 (9.2) 更新 policy，Critic 根据某种算法更新 value function. 这就是 **Actor-Critic** 方法的基本思想.

在不同的算法中 Critic 的更新方式不同，即更新 action value q 的方式不同. 如果使用 MC 那么对应的算法就称为 REINFORCE 或者 Monte Carlo policy gradient. 如果使用 TD 的方法，那么对应的算法就称为 Actor-Critic. 在 TD method 中最简单的就是使用 Q-learning 来更新 $Q^\pi(s,a)$，称之为 **Q-Value based Actor-Critic (QAC)**.

Algorithm 15 Q-Value based Actor-Critic (QAC)

1: **Aim:** Search for an optimal policy by maximizing $J(\theta)$.
2: **for** each time step t in each episode **do**
3: Generate a_t following $\pi(a \mid s_t,\theta_t)$, observe r_{t+1}, s_{t+1}, and then generate a_{t+1} following $\pi(a \mid s_{t+1},\theta_t)$. ▷ Generate the experience sample $(s_t,a_t,r_{t+1},s_{t+1},a_{t+1})$ for SARSA
4: **Critic (value update):**

$$w_{t+1} = w_t + \alpha_w\left[r_{t+1} + \gamma q(s_{t+1},a_{t+1},w_t) - q(s_t,a_t,w_t)\right]\nabla_w q(s_t,a_t,w_t)$$

5: **Actor (policy update):**

$$\theta_{t+1} = \theta_t + \alpha_\theta \nabla_\theta \ln \pi(a_t \mid s_t,\theta_t) q(s_t,a_t,w_{t+1})$$

6: **end for**

Note 9.1. ① *QAC* 中 *Critic* 使用 *SARSA* 型 *TD* 更新+ *value function approximation* ② *Actor* 即 *policy update algorithm* ③ 由于动作由当前策略采样，因此 *QAC* 是 *on-policy* ④ 随机策略本身提供探索，因此不必额外使用 *ϵ-greedy* ⑤ 但随机策略不保证探索充分，实际常加入 *entropy regularization* $J_{actor} = \mathbb{E}[\log \pi_\theta(a_t \mid s_t)\hat{A}_t + \beta\mathcal{H}(\pi_\theta(\cdot \mid s_t))]$.

9.2 Advantage Actor-Critic (A2C)

Advantage Actor-Critic (A2C): Baseline

Advantage actor-critic (A2C) 是 QAC 的一个推广，因为名称有两个 "a" 所以简称 A2C. 其核心 idea 是引入一个 baseline / 基准线以减小估计的方差:

$$\nabla_\theta J(\theta) = \mathbb{E}_{S\sim\eta, A\sim\pi}\left[\nabla_\theta \ln \pi(A|S,\theta_t)(q_\pi(S,A) - b(S))\right] \tag{9.3}$$

其中 $b(S)$ 为 S 的标量函数，被称为 baseline，其必须具备两个性质:

① **Valid:** 引入 baseline 不会对梯度 $\nabla_\theta J(\theta)$ 的值造成影响.

② **Useful:** 引入 baseline 可以降低 variance，从而提高采样结果稳定性.

Baseline Property

Proposition 9.1 (Valid Baseline)**.** 引入 *baseline* 不会对 $\nabla_\theta J(\theta)$ 的值造成影响.

Proof.

$$\begin{aligned}\mathbb{E}_{S\sim\eta,A\sim\pi}\left[\nabla_\theta \ln\pi(A|S,\theta_t)b(S)\right] &= \sum_{s\in\mathcal{S}}\eta(s)\sum_{a\in\mathcal{A}}\pi(a|s,\theta_t)\nabla_\theta\ln\pi(a|s,\theta_t)b(s) = \sum_{s\in\mathcal{S}}\eta(s)\sum_{a\in\mathcal{A}}\nabla_\theta\pi(a|s,\theta_t)b(s)\\ = \sum_{s\in\mathcal{S}}\eta(s)b(s)\sum_{a\in\mathcal{A}}\nabla_\theta\pi(a|s,\theta_t) &= \sum_{s\in\mathcal{S}}\eta(s)b(s)\nabla_\theta\sum_{a\in\mathcal{A}}\pi(a|s,\theta_t) = \sum_{s\in\mathcal{S}}\eta(s)b(s)\nabla_\theta 1 = 0.\end{aligned} \tag{9.4}$$

因此

$$\mathbb{E}_{S\sim\eta,A\sim\pi}\left[\nabla_\theta\ln\pi(A|S,\theta_t)q_\pi(S,A)\right] = \mathbb{E}_{S\sim\eta,A\sim\pi}\left[\nabla_\theta\ln\pi(A|S,\theta_t)(q_\pi(S,A)-b(S))\right]. \tag{9.5}$$

□

Proposition 9.2 (Useful Baseline)**.** 引入 适当选择的 *baseline* 可以降低 *variance*，提高采样结果稳定性.

Proof. 下面证明 $b(S)$ 会对 $\mathrm{var}(X)$ 造成影响，为此我们考虑 $\mathrm{tr}[\mathrm{var}(X)] = \mathbb{E}[X^TX] - \bar{x}^T\bar{x}$，就有

$$\mathbb{E}[X^TX] = \mathbb{E}\left[(\nabla_\theta\ln\pi)^T(\nabla_\theta\ln\pi)(q(S,A)-b(S))^2\right] = \mathbb{E}\left[\|\nabla_\theta\ln\pi\|^2(q(S,A)-b(S))^2\right]. \tag{9.6}$$

□

因此可以看出，方差较小的算法会使得在采样时采样得到的 x 更接近 $\mathbb{E}[X]$，所以我们希望通过改变 baseline 来尽量降低方差，甚至如果存在，我们希望找到一个最优的 baseline 来使得方差最小化.

Optimal Baseline: 实际上最优的 baseline 为:

$$b^*(s) = \frac{\mathbb{E}_{A\sim\pi}[\|\nabla_\theta\ln\pi(A|s,\theta_t)\|^2 q(s,A)]}{\mathbb{E}_{A\sim\pi}[\|\nabla_\theta\ln\pi(A|s,\theta_t)\|^2]}. \tag{9.7}$$

但是由于其形式复杂，在算法中通常使用如下形式的 baseline:

$$b(s) = \mathbb{E}_{A\sim\pi}[q(s,A)] = v_\pi(s). \tag{9.8}$$

Advantage Actor-Critic (A2C) Algorithm

根据前面的分析，我们令 $b(s) = v_\pi(s)$，这样得到的 gradient-ascent algorithm 为

$$\theta_{t+1} = \theta_t + \alpha\mathbb{E}\left[\nabla_\theta\ln\pi(A|S,\theta_t)[q_\pi(S,A)-v_\pi(S)]\right] \triangleq \theta_t + \alpha\mathbb{E}\left[\nabla_\theta\ln\pi(A|S,\theta_t)\delta_\pi(S,A)\right] \tag{9.9}$$

其中 $\delta_\pi(S,A) \triangleq q_\pi(S,A) - v_\pi(S)$ 称为优势函数 (advantage function). 因为根据定义 $v_\pi(S) = \sum_a \pi(a|S)q_\pi$，若 $\delta_\pi > 0$，就说明此 action 比平均的 action 要好，是有优势的.

据此得到梯度上升算法为:

$$\begin{aligned}\theta_{t+1} &= \theta_t + \alpha\nabla_\theta\ln\pi(a_t|s_t,\theta_t)[q_t(s_t,a_t)-v_t(s_t)] = \theta_t + \alpha\nabla_\theta\ln\pi(a_t|s_t,\theta_t)\delta_t(s_t,a_t)\\ &= \theta_t + \alpha\frac{\nabla_\theta\pi(a_t|s_t,\theta_t)}{\pi(a_t|s_t,\theta_t)}\delta_t(s_t,a_t) = \theta_t + \alpha\underbrace{\left(\frac{\delta_t(s_t,a_t)}{\pi(a_t|s_t,\theta_t)}\right)}_{\text{step size}}\nabla_\theta\pi(a_t|s_t,\theta_t)\end{aligned} \tag{9.10}$$

同 Lecture 8 中 Note 8.5 一样，step size 此时是探索 (exploration) 和利用 (exploitation) 的平衡. 并且由于我们更关心相对好坏，即 value 的相对值，因此这一节的 δ_t 较上一节的 q_t 更合理.

由于 $\delta_\pi(S,A) \triangleq q_\pi(S,A) - v_\pi(S)$ 的计算同时涉及到 action 和 state value，由于在期望上有:

$$\mathbb{E}[q_\pi(S,A)-v_\pi(S)|S=s_t,A=a_t] = \mathbb{E}\left[R+\gamma v_\pi(S')-v_\pi(S)|S=s_t,A=a_t\right]. \tag{9.11}$$

因此实际中一个小的技巧是将 δ_t 进行替换:

$$\delta_t = q_t(s_t,a_t) - v_t(s_t) \to r_{t+1} + \gamma v_t(s_{t+1}) - v_t(s_t). \tag{9.12}$$

此时 δ_t 只需要一个神经网络近似 v_t 即可，而原先需要两个网络分别近似 q_t 和 v_t.

Advantage Actor-Critic (A2C) Algorithm

Algorithm 16 Advantage Actor-Critic (A2C) or TD Actor-Critic(on-policy)

1: **Aim:** Search for an optimal policy by maximizing $J(\theta)$.
2: **for** each time step t in each episode **do**
3: Generate a_t following $\pi(a \mid s_t, \theta_t)$ and then observe r_{t+1}, s_{t+1}.
4: **TD error (advantage function):**

$$\delta_t = r_{t+1} + \gamma v(s_{t+1}, w_t) - v(s_t, w_t)$$

5: **Critic (value update):**

$$w_{t+1} = w_t + \alpha_w \delta_t \nabla_w v(s_t, w_t)$$

6: **Actor (policy update):**

$$\theta_{t+1} = \theta_t + \alpha_\theta \delta_t \nabla_\theta \ln \pi(a_t \mid s_t, \theta_t)$$

7: **end for**

9.3 Off-policy Actor-Critic

Introduction

在计算 policy gradient $\nabla_\theta J(\theta) = \mathbb{E}_{S\sim\eta, A\sim\pi}[*]$ 时，期望的理论计算需要使用 $A \sim \pi$，即动作是由当前策略采样得到的，因此 behavior policy 和 target policy 相同，故 QAC 和 A2C 都是 on-policy 算法. 为将算法转换为 off-policy 的，一种常用方法是使用 importance sampling[a].

[a] Importance sampling 也可以在别的地方例如 MC、TD 使用

Importance Sampling

Motivating Example. 根据大数定律，为估计某随机变量 X 的期望 $\mathbb{E}(X)$ 可多次采样得到 samples $\{x_i\}$ 然后估算:

$$\bar{x} = \sum_{i=1}^{n} \frac{1}{n} x_i \to \mathbb{E}_{X\sim p_0}[X]. \tag{9.13}$$

其中 p_0 为 X 的分布. 在实际中我们希望依据一个已知但不同于 p_0 的分布 p_1 进行采样，但最终仍用于估算 $\mathbb{E}(X)$. 这对应了 off-policy 中使用与 target policy 不同的 behavior policy 产生数据轨迹，但最终估计的是 target policy.

Importance Sampling. 实际上可以通过简单的变量替换实现使用已知的 p_1 代替 p_0 进行采样:

$$\mathbb{E}_{X\sim p_0}[X] = \sum_x p_0(x) x = \sum_x p_1(x) \underbrace{\frac{p_0(x)}{p_1(x)}}_{f(x)} x = \mathbb{E}_{X\sim p_1}[f(X)]. \Rightarrow \bar{f} \triangleq \frac{1}{n}\sum_{i=1}^{n} f(x_i), \quad \text{where } x_i \sim p_1. \tag{9.14}$$

最后得到

$$\mathbb{E}_{X\sim p_0}[X] \approx \bar{f} = \frac{1}{n}\sum_{i=1}^{n} f(x_i) = \frac{1}{n}\sum_{i=1}^{n} \frac{p_0(x_i)}{p_1(x_i)} x_i. \tag{9.15}$$

其中 $\frac{p_0(x_i)}{p_1(x_i)}$ 被称为重要性权重 (importance weight).

Note 9.2. ① 一个直观的解释是若某样本点 x_i 在 p_0 下很容易被采到但是在 p_1 下很难被采到，那么就要很珍惜这个样本点，直观上来说这个样本点的权重应该比较大，这与 $\frac{p_0(x_i)}{p_1(x_i)}$ 此时很大刚好对应.

② *Importance sampling* 的应用逻辑为：*(1)* 若可以使用 p_0 进行采样那么直接使用 $\bar{x} = \sum_{i=1}^{n} \frac{1}{n} x_i \to \mathbb{E}_{X\sim p_0}[X]$ 即可 *(2)* 若只能根据 p_1 进行采样得到 $\{x_i\}$ 并已知这些离散点的 $p_0(x_i)$ [a] *(3)* 由于定义式 $\mathbb{E}_{X\sim p_0}[X] = \sum_x p_0(x)x$ 需要知道所有点的 p_0 因此无法直接利用 *(4)* 因此最终只能通过转化成 p_1 下有限个点来计算 p_0 的期望.

[a] 例如使用一个神经网络逼近 p_0，但是此时 p_0 都没有表达式.

Off-policy Policy Gradient Algorithm

考虑 off-policy 情形，即 behavior policy 不同于 target policy π. 设此时 behavior policy 为 β. 此时 metric 写为:

$$J(\theta) = \sum_{s\in\mathcal{S}} d_\beta(s) v_\pi(s) = \mathbb{E}_{S\sim d_\beta}[v_\pi(S)], \tag{9.16}$$

其中 d_β 为 policy β 的 stationary distribution.
直接给出 $J(\theta)$ 的梯度如下:

$$\nabla_\theta J(\theta) = \mathbb{E}_{S\sim\rho, A\sim\beta}\left[\frac{\pi(A|S,\theta)}{\beta(A|S)} \nabla_\theta \ln \pi(A|S,\theta) q_\pi(S,A)\right], \tag{9.17}$$

其中 $\gamma \in (0,1)$ 为 discount factor, ρ 为 state distribution.
加上 baseline $b(s)$，得到:

$$\nabla_\theta J(\theta) = \mathbb{E}_{S\sim\rho, A\sim\beta}\left[\frac{\pi(A|S,\theta)}{\beta(A|S)} \nabla_\theta \ln \pi(A|S,\theta) \left(q_\pi(S,A) - b(S)\right)\right]. \tag{9.18}$$

根据前面的结构，令 $b(s) = v_\pi(s)$，得到

$$\nabla_\theta J(\theta) = \mathbb{E}\left[\frac{\pi(A|S,\theta)}{\beta(A|S)} \nabla_\theta \ln \pi(A|S,\theta) \left(q_\pi(S,A) - v_\pi(S)\right)\right]. \tag{9.19}$$

对应的 stochastic 版本即:

$$\theta_{t+1} = \theta_t + \alpha_\theta \frac{\pi(a_t|s_t,\theta_t)}{\beta(a_t|s_t)} \nabla_\theta \ln \pi(a_t|s_t,\theta_t) \left(q_t(s_t,a_t) - v_t(s_t)\right). \tag{9.20}$$

与 on-policy 类似地有

$$q_t(s_t,a_t) - v_t(s_t) \approx r_{t+1} + \gamma v_t(s_{t+1}) - v_t(s_t) \triangleq \delta_t(s_t,a_t). \tag{9.21}$$

如此得到算法

$$\theta_{t+1} = \theta_t + \alpha_\theta \frac{\pi(a_t|s_t,\theta_t)}{\beta(a_t|s_t)} \nabla_\theta \ln \pi(a_t|s_t,\theta_t) \delta_t(s_t,a_t) = \theta_t + \alpha_\theta \left(\frac{\delta_t(s_t,a_t)}{\beta(a_t|s_t)}\right) \nabla_\theta \pi(a_t|s_t,\theta_t). \tag{9.22}$$

Algorithm 17 Off-policy Actor-Critic Based on Importance Sampling

1: **Initialization:** A given behavior policy $\beta(a \mid s)$. A target policy $\pi(a \mid s, \theta_0)$ where θ_0 is the initial parameter vector. A value function $v(s, w_0)$ where w_0 is the initial parameter vector.
2: **Aim:** Search for an optimal policy by maximizing $J(\theta)$.
3: **for** each time step t in each episode **do**
4: Generate a_t following $\beta(\cdot \mid s_t)$ and then observe r_{t+1}, s_{t+1}.
5: **TD error (advantage function):**

$$\delta_t = r_{t+1} + \gamma v(s_{t+1}, w_t) - v(s_t, w_t)$$

6: **Critic (value update):**

$$w_{t+1} = w_t + \alpha_w \frac{\pi(a_t \mid s_t, \theta_t)}{\beta(a_t \mid s_t)} \delta_t \nabla_w v(s_t, w_t)$$

7: **Actor (policy update):**

$$\theta_{t+1} = \theta_t + \alpha_\theta \frac{\pi(a_t \mid s_t, \theta_t)}{\beta(a_t \mid s_t)} \delta_t \nabla_\theta \ln \pi(a_t \mid s_t, \theta_t)$$

8: **end for**

9.4 Deterministic Actor-Critic (DPG)

Introduction: Continuous Action Space

前面介绍的所有的算法都有 $\pi(a|s,\theta)>0$，也就是说都带有 exploration 性，都是 stochastic 策略. 但是 $\pi(a|s,\theta)>0$ 隐含假设了 action space 是有限的，因为 $\sum_a \pi(a|s;\theta)=1$. 因此对于 continuous action space 还需要额外处理.

Deterministic Actor-Critic (DPG)

下面仅考虑 deterministic 的情形，此时重新定义一个 deterministic policy:

$$a=\mu(s,\theta)\doteq\mu(s). \tag{9.23}$$

这与 $\pi(a|s;\theta)$ 不同的是直接输出 action，而不是输出采取某些 action 对应的概率值.

Note 9.3. ① μ 是 *state space* 到 *action space* 的映射：$\mu:\mathcal{S}\to\mathcal{A}$. ② μ 可用参数为 θ 的神经网络替代，输入 s 输出 a.

与之前的推导过程类似，首先定义 discounted case 下的 scalar metric:

$$J(\theta)=\mathbb{E}[v_\mu(s)]=\sum_{s\in\mathcal{S}} d_0(s)v_\mu(s), \tag{9.24}$$

其中 $d_0(s)$ 为概率分布，满足 $\sum_{s\in\mathcal{S}} d_0(s)=1$，最简单的情形是与 μ 不相关 (例如只有某 s_0 的 $d_0(s_0)=1$，或 behavior policy 的 stationary distribution).
此处直接给出 discounted case 下的梯度表达式为

$$\nabla_\theta J(\theta)=\sum_{s\in\mathcal{S}}\rho_\mu(s)\nabla_\theta\mu(s)\left(\nabla_a q_\mu(s,a)\right)|_{a=\mu(s)}=\mathbb{E}_{S\sim\rho_\mu}\left[\nabla_\theta\mu(S)\left(\nabla_a q_\mu(S,a)\right)|_{a=\mu(S)}\right], \tag{9.25}$$

其中 $\gamma\in(0,1)$ 为 discount factor, ρ_μ 为 state distribution. 注意在此梯度的表达式中最后已经没有 a，即对 target policy 的更新已经不再和 behavior policy 相关，因此天然是 off-policy 的.
再代入梯度上升算法中，得到:

$$\theta_{t+1}=\theta_t+\alpha_\theta\mathbb{E}_{S\sim\rho_\mu}\left[\nabla_\theta\mu(S)\left(\nabla_a q_\mu(S,a)\right)|_{a=\mu(S)}\right]. \tag{9.26}$$

那么得到实际使用的 stochastic 版本为:

$$\theta_{t+1}=\theta_t+\alpha_\theta\nabla_\theta\mu(s_t)\left(\nabla_a q_\mu(s_t,a)\right)|_{a=\mu(s_t)}. \tag{9.27}$$

Algorithm 18 Deterministic Actor-Critic Algorithm

1: **Initialization:** A given behavior policy $\beta(a\mid s)$. A deterministic target policy $\mu(s,\theta_0)$ where θ_0 is the initial parameter vector. A value function $v(s,a,w_0)$ where w_0 is the initial parameter vector.
2: **Aim:** Search for an optimal policy by maximizing $J(\theta)$.
3: **for** each time step t in each episode **do**
4: Generate a_t following β and then observe r_{t+1},s_{t+1}.
5: **TD error:**

$$\delta_t=r_{t+1}+\gamma q(s_{t+1},\mu(s_{t+1},\theta_t),w_t)-q(s_t,a_t,w_t)$$

6: **Critic (value update):**

$$w_{t+1}=w_t+\alpha_w\delta_t\nabla_w q(s_t,a_t,w_t)$$

7: **Actor (policy update):**

$$\theta_{t+1}=\theta_t+\alpha_\theta\nabla_\theta\mu(s_t,\theta_t)\left(\nabla_a q(s_t,a,w_{t+1})\right)\Big|_{a=\mu(s_t)}$$

8: **end for**

Note 9.4. ① *DPG* 算法是 *off-policy* 的，*behavior policy* β 可以与 *target policy* μ 不同. ② β 也可以是 μ+*noise*，这样就有了探索性 (此时就是 *on-policy* 的实现方式). ③ 当使用线性函数逼近 $q(s,a,w)$ 时，即 $q(s,a,w)=\phi^T(s,a)w$，此时算法为 *DPG*. ④ 当使用神经网络逼近 $q(s,a,w)$ 时，被称为 *deep deterministic policy gradient (DDPG)*.